



MIC AIML Department Recruitment

Welcome to the **MIC's AIML Department Recruitment** Challenge!

This challenge is your chance to show us how you think, build, and problem-solve. **Pick a track that excites you, dig in, and build something you are proud of.** We are not just looking for working code; we are looking for people who genuinely understand what they built and why.

Duration: __ *August* to __ *August*

Challenge Overview

- ❖ Choose **ONE track** from **Computer Vision, NLP + Chatbot, or Data Science & Visualisation**.
- ❖ Complete the parts assigned to your academic year (see the **Requirements table** under each track).
- ❖ Bonus stretch goals are **optional** - they will not affect eligibility but can earn you extra credit and **stand out to reviewers**.
- ❖ **Submit before the deadline** with all required materials (see **Submission Requirements**).

Important Guideline: Choose only one track. Focus on doing it well rather than attempting multiple tracks.

1. Computer Vision

AI Traffic Police

Build a computer vision system that helps traffic authorities monitor roads, detect incidents, and improve traffic management.

Part 1 – Foundations

- Classify vehicle types (car, bus, bike, truck)
- Count vehicles from images/videos

Part 2 – Detection

- Detect basic traffic violations
- Detect emergency vehicles

Part 3 – Advanced

- Accident detection
- Number plate recognition

Stretch Goals (Optional)

- Analytics dashboard
- Multi-camera support
- Speed optimization
- Heatmaps of traffic density

Requirements by Academic Year

Year	What to Complete
1st Year	Part 1 ONLY
2nd Year	Part 2 ONLY
3rd & 4th Year	Part 3 ONLY

Recommended Datasets & Resources

(Suggestions only — not mandatory)

- [COCO](#)
- [Roboflow Universe](#)
- [BDD100K](#)
- Relevant Kaggle datasets

2. NLP + Chatbot

AI Recruiter

Build an AI-powered recruitment assistant that analyzes resumes, matches candidates with job roles, and assists recruiters in hiring decisions.

Part 1 – Extraction

Input: Resume

Output:

- Skills
- Technologies
- Languages

Part 2 – Matching

The system should be able to:

- Extract skills from resumes
- Suggest suitable job roles
- Match candidates to job descriptions

Part 3 – RAG Recruiter

The system should be able to:

- Understand resumes in depth
- Analyze job descriptions
- Answer questions about candidate-job fit
- Provide reasoning behind its recommendations

Stretch Goals (Optional)

- Multilingual support
- Voice interface
- Conversation memory
- Improved retrieval accuracy

Requirements by Academic Year

Year	What to Complete
1st Year	Part 1 ONLY
2nd Year	Part 1 + Part 2 ONLY
3rd & 4th Year	Part 3 ONLY

Recommended Datasets & Resources *(Suggestions only — not mandatory)*

- [Hugging Face Datasets](#)
- Resume datasets (Kaggle)
- Other relevant Kaggle datasets

3. Data science and Visualisation

AI Travel Analyst

Build a data-driven solution to analyze flight prices, uncover the factors that drive costs, and help travelers make smarter decisions.

Part 1 – Exploration

Given a flight dataset containing airline, source, destination, duration, stops, and departure time:

- Clean and preprocess the data
- Create at least 5 meaningful visualizations
- Identify major factors affecting flight prices
- Present insights and recommendations

Part 2 – Modeling

- Perform feature engineering
- Build a prediction model
- Train and evaluate the model
- Explain the key features driving predictions

Part 3 – Choose ONE

- Cheapest Booking Time Analysis
- Flight Price Forecasting
- Flight Recommendation System

Stretch Goals (Optional)

- Interactive dashboard
- Model explainability (e.g., SHAP)
- Forecast comparison across models
- Recommendation system

Requirements by Academic Year

Year	What to Complete
1st Year	Part 1 ONLY
2nd Year	Part 1 + Part 2 ONLY
3rd & 4th Year	Part 1 + Part 2 + Part 3 ONLY

Recommended Datasets & Resources (*Suggestions only — not mandatory*)

- Relevant Kaggle datasets
- Government Open Data portals
- [UCI Machine Learning Repository](#)

Tools & Resources

You are free to build with any of the following:

Libraries & Frameworks:

TensorFlow · PyTorch · Scikit-learn · OpenCV · Hugging
Face · LangChain · FastAPI · Streamlit · Gradio · Ultralytics YOLO · any other open-source library

AI Tools:

ChatGPT · Claude · Gemini · GitHub Copilot · Any LLM model

Important: You may use any of the above freely - but you must fully understand and be able to explain every part of your submission. Using an AI tool to generate code you cannot explain will hurt your evaluation, not help it.

README Requirement

Every submission must include a README covering:

- Project Overview
- Problem Statement
- Installation Instructions
- Dataset Used
- Methodology
- Technologies Used
- Results
- Challenges Faced
- Future Improvements
- Screenshots (if applicable)

A strong README is often what separates a good submission from a great one — treat it as part of the project, not an afterthought.

Submission Requirements

Your submission should include:

- **GitHub Repository** (public)
- **README** (see requirements above)
- **Source Code**
- **Requirements file** (requirements.txt, package.json, etc.)
- **Documentation**
- **Demo Video** (*optional but recommended*)

Tips

- A clean, incremental Git commit history is viewed positively - it shows how you built the project, not just the result.
- We evaluate projects holistically. A simple, well-explained solution is preferred over a complex project that you cannot justify.

Evaluation Rubric

Your submission will be evaluated on the following criteria:

Criterion	Weight	What We're Looking For
Understanding & Explanation	25%	Can you clearly explain what your project does and why you made the choices you made?
Technical Implementation	25%	Does the solution work correctly and handle the core requirements of your track?
Creativity & Innovation	20%	Did you bring original thinking, unique features, or a fresh approach to the problem?
Code Quality	15%	Is your code clean, readable, and well-structured?
Documentation	15%	Is your README clear, complete, and easy for someone else to follow?

Note: Project Completeness and Engineering Decisions (e.g., why you chose a particular model, architecture, or library) are considered within Technical Implementation and Understanding & Explanation above.

FAQs

Is this an individual or team challenge?

This is an **individual** challenge. Every submission must be your own work.

Can I participate in more than one track?

No. Choose **one track** and focus on doing it well.

Can I choose a different project idea?

No. Please choose one of the provided challenge tracks and complete the required tasks for your academic year.

Can I complete higher-year tasks?

Yes, but only the required part for your academic year is expected. Additional work may help display your skills but is not mandatory.

Do I have to use the recommended datasets?

No. They are suggestions only. You are free to use any appropriate dataset unless otherwise specified.

Can I use my own dataset?

Yes, provided it is relevant to the challenge.

Can I use pretrained models?

Yes. However, we value how you build around it and how well you understand your implementation. **Pretrained AI models that directly solve the core task of the challenge are not encouraged** though.

Which programming language should I use?

Any programming language is allowed.

Can I build a web app instead of a notebook?

Yes. A notebook, desktop application, CLI, API, or web application are all acceptable if they show the required functionality.

Will AI-generated code reduce my score?

No. Using AI is not penalized. **But submitting code you cannot explain will negatively impact your evaluation.**

Can I submit it after the deadline?

Late submissions may **not be accepted** unless specified for a valid reason beforehand.

Can I update my GitHub repository after submitting it?

No. Only the version available at the submission deadline will be evaluated.

What if I cannot finish everything?

That is okay. We evaluate the quality of your work and your understanding—not just the number of features you implement. **A polished, well-explained project is better than an unfinished ambitious one.**

Can I copy code from GitHub?

You may refer to open-source learning projects, but your submission should be your own work. If you reuse code, ensure you comply with its license and understand what it does.

Interview Expectations

Shortlisted candidates may be asked to explain:

- Their architecture
- Model selection
- Dataset choice
- Preprocessing steps
- Evaluation metrics
- Challenges faced
- Improvements

Expect a **technical discussion**, not just a demo. Be ready to walk through your reasoning, not just your results.

Important Notes

- **Originality is valued more than model complexity.** A well-understood simple solution beats a black-box complex one.
- **Downloading a pretrained model and running inference alone is unlikely to score well.** Show us your own thoughts.
- **Clean code is preferred over clever, complicated code.**
- **Good documentation matters** — it is part of the evaluation, not an afterthought.
- **Understanding your implementation matters more than chasing the highest accuracy.**

Final Checklist Before You Submit

- Chosen exactly one track
- Completed all parts required for your academic year
- Code is clean and well-organized
- README includes all required sections
- Repository includes source code + requirements file
- You can explain every part of your submission
- (Optional) Attempted a stretch goal
- (Optional) Recorded a demo video

BUILD BOLDLY. LEARN SOMETHING NEW. MOST IMPORTANTLY, ENJOY THE PROCESS.

WE LOOK FORWARD TO SEEING YOUR IDEAS COME TO LIFE.